AIGC v1.0 GitHub project tracker

Project title

AIGC v1.0 — Workflow Governance Without Breaking the SDK Boundary

Project objective

Ship v1.0.0 as:
• deterministic invocation governance
• first-class workflow-governance primitives
• Bedrock/A2A-ready integration adapters
• stronger adoption surface
• no hosted platform creep

This stays aligned with the current repo boundary where the host owns orchestration and model calls, while AIGC owns policy loading, ordered governance checks, and audit generation. [obj] [obj]

———

PR tracker

PR-01

Name: ADR + v1.0 boundary contract
Branch: feat/v1.0-01-adr-boundary-contract
Depends on: none

Scope
• Add docs/decisions/ADR-0011-workflow-governance-v1.md
• Add docs/SDK_BOUNDARY.md
• Add docs/PUBLIC_API_STABILITY.md
• Add docs/MIGRATION_0_3_3_TO_1_0.md
• Add v1.0 sections to RELEASE_GATES.md
• Update README.md, PROJECT.md, CHANGELOG.md, docs/PUBLIC_INTEGRATION_CONTRACT.md links as needed
• Add doc parity updates for new canonical docs

Acceptance criteria
• AIGC v1.0 scope is explicit
• workflow governance is defined as an SDK primitive layer, not a platform
• non-goals are explicit
• public API stability policy is explicit
• migration framing is explicit
• no runtime behavior changes in this PR

Test targets
• markdown lint passes
• doc parity script updated and passing
• no runtime test delta required

———

PR-02

Name: Workflow audit correlation schema
Branch: feat/v1.0-02-workflow-audit-schema
Depends on: PR-01

Scope
• Add additive workflow-level audit fields or a companion workflow artifact schema

• Add workflow correlation support to compliance/export paths
• Preserve one artifact per invocation attempt
• Keep invocation artifact contract backward-compatible

Acceptance criteria
• existing 0.3.3 artifacts remain valid
• no new required fields on per-invocation artifacts
• workflow-level evidence is additive only
• schema docs updated

Test targets
• 15–20 schema compatibility tests
• artifact backward-compat tests
• golden replay unchanged for legacy invocation paths

———

PR-03

Name: GovernanceSession core primitive
Branch: feat/v1.0-03-governance-session
Depends on: PR-02

Scope
• Add GovernanceSession
• step registration
• workflow/session identifiers
• step-to-artifact correlation
• workflow finalize artifact
• single-use / fail-closed session rules

Acceptance criteria
• one workflow can govern many invocations
• session finalization produces deterministic workflow evidence
• existing enforce_invocation, split APIs, and @governed still work unchanged
• no hidden orchestration engine introduced

Test targets
• 25–35 unit tests
• concurrent independent sessions
• finalize-on-failure behavior
• immutability/lifecycle tests

———

PR-04

Name: Workflow policy DSL extension
Branch: feat/v1.0-04-workflow-dsl
Depends on: PR-03

Scope
• Extend DSL with:
• workflow.max_steps
• workflow.max_total_tool_calls
• workflow.allowed_transitions
• workflow.required_sequence
• workflow.allowed_agent_roles

• workflow.handoffs
• workflow.escalation
• workflow.protocol_constraints

Acceptance criteria
• old policies remain valid
• new workflow fields are schema-validated
• DSL remains declarative
• composition behavior defined for workflow sections

Test targets
• 20–30 DSL validation tests
• composition tests
• invalid workflow policy tests

———

PR-05

Name: Workflow enforcement engine
Branch: feat/v1.0-05-workflow-enforcement
Depends on: PR-04

Scope
• Enforce step sequence
• enforce transition rules
• enforce cumulative tool budgets
• enforce handoff permissions
• enforce workflow-level pre/postconditions
• wrap existing invocation kernel, do not replace it

Acceptance criteria
• invalid step order is blocked
• over-budget workflows are blocked
• disallowed handoffs are blocked
• per-step split enforcement invariants still hold

Test targets
• 30–40 workflow enforcement tests
• integration tests for step ordering
• golden replays for session-aware paths if applicable

———

PR-06

Name: Agent identity and capability manifests
Branch: feat/v1.0-06-agent-manifests
Depends on: PR-05

Scope
• Add AgentIdentity
• Add AgentCapabilityManifest
• validate workflow participants against policy

Acceptance criteria
• agent roles are validated

• capability mismatches fail closed
• manifest versioning rules documented

Test targets
• 15–20 manifest validation tests

———

PR-07

Name: Bedrock workflow adapter
Branch: feat/v1.0-07-bedrock-adapter
Depends on: PR-06

Scope
• Add optional Bedrock adapter module
• normalize Bedrock supervisor/collaborator workflow events into AIGC workflow steps
• keep AWS-specific logic out of core enforcement

Acceptance criteria
• adapter is optional
• no required AWS dependency in core path
• supervisor/collaborator roles map cleanly into workflow policy
• docs clearly position adapter as integration layer

Test targets
• 15–20 adapter tests
• mocked event normalization tests
• no live AWS calls

———

PR-08

Name: A2A workflow adapter
Branch: feat/v1.0-08-a2a-adapter
Depends on: PR-06

Scope
• Add optional A2A adapter module
• agent-card normalization
• handoff envelope validation
• session/protocol metadata capture
• remote-agent handoff governance hooks

Acceptance criteria
• adapter is optional
• no transport ownership in SDK
• A2A-specific metadata maps into generic workflow primitives

Test targets
• 15–20 adapter tests
• mocked JSON-RPC/A2A envelope tests

———

PR-09

Name: Human review and escalation checkpoints
Branch: feat/v1.0-09-escalation

Depends on: PR-05

Scope
• Add escalation states
• add approval token/evidence model
• allow workflow pause/resume under governance

Acceptance criteria
• review-required path is deterministic
• approval/denial is auditable
• no hidden bypass path

Test targets
• 20–25 escalation tests
• pause/resume lifecycle tests

———

PR-10

Name: Content-validation hook layer
Branch: feat/v1.0-10-validator-hooks
Depends on: PR-05

Scope
• Add stable validator hook API
• support plugging in external semantic/content validators
• do not turn AIGC into NeMo/Guardrails

Acceptance criteria
• hook API is documented and stable
• core pipeline order is preserved
• optional content validators can fail closed

Test targets
• 15–20 hook tests
• ordering tests
• metadata propagation tests

———

PR-11

Name: Workflow CLI and export tools
Branch: feat/v1.0-11-workflow-cli
Depends on: PR-03, PR-05

Scope
• aigc workflow trace
• aigc workflow export
• aigc workflow lint
• timeline/session reconstruction

Acceptance criteria
• operators can inspect a full workflow
• exports correlate workflow and invocation evidence
• CLI docs updated

Test targets
• 20–25 CLI tests

• fixture-based export tests

———

PR-12

Name: Starter workflow policy packs
Branch: feat/v1.0-12-starter-workflows
Depends on: PR-04, PR-05

Scope
• starter policies for:
• supervisor/collaborator workflow
• review-required workflow
• remote handoff workflow
• source-required workflow
• tool-budget workflow

Acceptance criteria
• all starter policies validate
• each starter has at least one pass/fail test
• clearly positioned as SDK starters, not vertical app packs

Test targets
• 10–15 starter policy tests

———

PR-13

Name: Integration recipes and quickstarts
Branch: feat/v1.0-13-recipes-quickstarts
Depends on: PR-07, PR-08, PR-11, PR-12

Scope
• Bedrock recipe
• A2A recipe
• FastAPI host recipe
• worker/queue recipe
• 15-minute quickstart

Acceptance criteria
• examples are runnable
• no example imports from aigc._internal
• docs align with actual public API

Test targets
• 10–15 example smoke tests

———

PR-14

Name: Public API and operations hardening
Branch: feat/v1.0-14-api-ops-hardening
Depends on: PR-01 through PR-13

Scope
• finalize PUBLIC_API_STABILITY.md
• add supported environments matrix

• add operations runbook references
• add troubleshooting updates
• update doc_parity_manifest.yaml

Acceptance criteria
• public API surface is frozen for 1.x
• supported import boundary is explicit
• doc parity manifest reflects 1.0.0
• migration guide is complete

Test targets
• doc parity passes
• packaging/install smoke tests
• public API snapshot tests

———

PR-15

Name: v1.0.0 release PR
Branch: release/v1.0.0
Depends on: all prior PRs

Scope
• version bump
• changelog
• final release notes
• final parity sweep
• release gate signoff

Acceptance criteria
• all v1.0 gates green
• docs parity green
• public API snapshot green
• workflow governance examples green
• migration docs published

Test targets
• full CI matrix
• public API snapshot
• packaging smoke
• example smoke
• workflow integration suite
• coverage still above gate

———

Suggested dependency graph

Use this order:
1. PR-01
2. PR-02
3. PR-03
4. PR-04
5. PR-05
6. PR-06
7. PR-07 and PR-08 in parallel
8. PR-09 and PR-10 in parallel

9. PR-11
10. PR-12
11. PR-13
12. PR-14
13. PR-15

This keeps the architecture-first discipline already visible in the current release process and release gates. 🫧

———

Draft file: docs/decisions/ADR-0011-workflow-governance-v1.md

ADR-0011: Workflow Governance in AIGC v1.0

Date: 2026-04-13
Status: Proposed
Owners: Neal

Context

AIGC `v0.3.3` is the current release on `develop`.

It already provides:

- deterministic invocation-boundary governance
- unified and split enforcement
- audit schema `v1.4`
- provenance metadata
- `AuditLineage`
- `ProvenanceGate`
- `RiskHistory`
- split enforcement as the default decorator execution model

At the same time, the repo's design narrative still places orchestration and model calls in the host application. AIGC owns policy loading, ordered governance checks, and audit artifact generation. It does not host agents, does not make model calls, and does not own business-state orchestration.

The strongest remaining architectural gap is workflow-level governance for multi-step agentic systems. Current workflow-aware features make lineage and provenance visible across invocations, but they do not yet provide first-class workflow primitives such as:

- workflow/session state
- step sequencing
- cumulative tool budgets
- explicit handoff governance
- workflow-level approval checkpoints
- workflow-level audit correlation

AIGC v1.0 needs to close that gap without turning the SDK into a platform.

Decision

Adopt workflow governance as a first-class SDK capability in `v1.0.0`.

AIGC v1.0 will remain an SDK and will not become a hosted runtime, orchestration framework, or vertical reference application. The release adds a thin workflow-governance layer on top of the existing invocation-governance kernel.

This workflow-governance layer includes:

1. `GovernanceSession`
2. workflow-aware policy DSL extensions
3. workflow enforcement over step order, handoffs, and aggregate budgets
4. agent identity and capability manifests
5. optional integration adapters for Bedrock-style multi-agent workflows and A2A-style remote agent handoffs
6. human review / escalation checkpoints
7. workflow trace and export utilities

Non-Goals

AIGC v1.0 does not:

- host agents or tools
- run an HTTP server
- own protocol transport implementations
- replace LangGraph, Bedrock AgentCore, or other orchestrators
- introduce business-specific reference architectures into the core SDK
- add provider lock-in
- add required dependencies on cloud SDKs in the core path

Architectural Boundary

Host application owns

- orchestration
- model calls
- tool execution
- business state
- persistence beyond audit artifact emission
- protocol transport runtime
- vertical application logic

AIGC owns

- policy loading and validation
- ordered invocation governance
- workflow/session governance primitives

- workflow step/handoff enforcement
- workflow-level evidence correlation
- audit artifact generation
- CLI export and trace utilities

Invariants Preserved

The following invariants remain non-negotiable:

1. Fail-closed behavior remains mandatory.
2. Gate ordering for invocation enforcement remains preserved.
3. Exactly one audit artifact is emitted per invocation attempt.
4. Audit schema evolution remains additive for existing invocation artifacts.
5. Unified mode remains supported while deprecated where already documented.
6. Workflow governance wraps the invocation kernel; it does not bypass it.
7. Public API additions must be explicit and versioned.

Options Considered

Option A: Keep AIGC invocation-only

Pros:

- lowest implementation risk
- no boundary expansion

Cons:

- leaves the most important architectural gap unresolved
- makes AIGC weaker for agentic systems

Option B: Add workflow governance primitives inside the SDK (chosen)

Pros:

- closes the most important missing governance layer
- preserves SDK identity
- maximizes adoption across internal, Bedrock, and A2A-style systems

Cons:

- requires careful boundary discipline
- expands policy DSL and evidence model

Option C: Turn AIGC into a platform

Pros:

- more end-to-end control

Cons:

- breaks the current SDK/host contract

- increases lock-in and scope
- dilutes adoption as a lightweight governance layer

Consequences

What becomes easier

- governing multi-step workflows
- constraining agent-to-agent handoffs
- correlating workflow-level evidence
- adding review checkpoints to high-risk flows
- adopting AIGC in Bedrock-style and A2A-style systems

What becomes harder

- maintaining a crisp SDK boundary
- evolving the public API carefully
- documenting workflow semantics clearly

Risks

- workflow scope creep into orchestration
- adapter logic leaking cloud-specific assumptions into core
- parity drift between docs, examples, and public API

Mitigations

- explicit boundary docs
- optional adapters only
- public API stability contract
- doc parity gates
- release gates for every workflow workstream

Contract Impact

Public API

Additive public API changes are expected for:

- `GovernanceSession`
- workflow CLI
- workflow DSL support
- optional integration adapter modules

Audit contract

Invocation artifact contract remains backward-compatible.
Workflow evidence is additive.

Docs

The following docs must be created or updated before release:

- `docs/SDK_BOUNDARY.md`
- `docs/PUBLIC_API_STABILITY.md`
- `docs/MIGRATION_0_3_3_TO_1_0.md`
- `RELEASE_GATES.md`
- `README.md`
- `PROJECT.md`
- `docs/PUBLIC_INTEGRATION_CONTRACT.md`
- `doc_parity_manifest.yaml`

———

Draft file: docs/SDK_BOUNDARY.md

AIGC SDK Boundary

This document defines what AIGC owns and what it intentionally leaves to the host application.

AIGC is a Python SDK for deterministic, fail-closed governance of AI behavior at runtime.

Its primary design center remains the invocation boundary. In `v1.0.0`, AIGC also adds workflow-governance primitives for multi-step agentic systems.
It does not become a platform.

What AIGC Does

AIGC owns:

- policy loading and validation
- ordered governance checks
- role, precondition, tool, schema, postcondition, and risk evaluation
- split enforcement across pre-call and post-call phases
- audit artifact generation for PASS and FAIL paths
- provenance and lineage primitives
- workflow/session governance primitives
- workflow step sequencing and handoff constraints
- workflow-level evidence correlation
- custom enforcement gates
- workflow trace and export tooling

At a high level:

- the host decides to act
- AIGC decides whether that action is policy-valid

- AIGC emits evidence of that decision

What AIGC Does Not Do

AIGC does not:

- make LLM calls
- run orchestration loops
- execute tools on the host's behalf
- host agents or tools
- provide an HTTP runtime
- persist application business state
- replace application authorization systems
- replace semantic content-safety systems
- replace full workflow engines or agent frameworks
- embed vertical business logic in the core SDK

If your system needs to decide *how* to orchestrate, it is the host's job.
If your system needs to decide *whether* the next step is allowed under declared policy, that is AIGC's job.

Host Responsibilities

The host application owns:

- orchestration
- model-provider integration
- protocol transport
- session persistence outside AIGC evidence artifacts
- tool execution
- human task routing outside approval evidence capture
- UI and operator interfaces
- domain/business logic
- downstream side effects

Examples:

- LangGraph graph construction
- Bedrock AgentCore runtime deployment
- A2A transport handling
- app-specific state machines
- CRM writes
- email sending
- case management logic

AIGC Responsibilities

AIGC owns the policy-governance contract around those actions.

Examples:

- is this role allowed?
- is the required session context present?
- has the review step already happened?
- is this handoff allowed?
- has this workflow exceeded its tool budget?
- does this output match its declared schema?
- must this step pause for human approval?
- what audit evidence must be emitted?

Invocation Boundary vs Workflow Boundary

Invocation boundary

AIGC's original and still-core surface:

- govern one invocation attempt
- emit one final artifact for that invocation attempt

Workflow boundary

AIGC v1.0 additive layer:

- govern multi-step sequences of invocation attempts
- correlate those invocation artifacts into workflow evidence
- enforce step order, budgets, handoffs, and escalation

Workflow governance wraps invocation governance.
It does not replace it.

Adapter Policy

Integration adapters may exist for:

- Bedrock multi-agent patterns
- A2A-style agent handoffs
- future workflow ecosystems

These adapters must:

- be optional
- preserve the generic core abstractions
- avoid pulling cloud-specific dependencies into the core path
- translate external workflow/protocol events into AIGC-native workflow governance primitives

Adapters are integration aids, not the product center.

One-Line Rule

AIGC governs actions. The host performs them.

———

Draft file: docs/PUBLIC_API_STABILITY.md

AIGC Public API Stability

This document defines the stability contract for the AIGC public API.

`v1.0.0` is the first release that establishes a formal public API stability promise for the `1.x` line.

Stability Promise for `1.x`

For all `1.x` releases:

- symbols exported from the supported public API are considered stable
- breaking changes to supported public API require a major version bump
- additive changes are allowed in minor and patch releases
- deprecations must be documented before removal
- undocumented internal modules are not covered by this contract

---

Supported Public API

The supported public API is:

- top-level exports from `aigc`
- documented public modules under `aigc/`
- documented CLI commands
- documented public policy DSL fields
- documented public audit artifact fields
- documented public example contracts where explicitly marked stable

Examples of supported public surfaces include:

- `enforce_invocation`
- `enforce_pre_call`
- `enforce_post_call`
- `AIGC`
- `@governed`
- `AuditLineage`
- `ProvenanceGate`
- `RiskHistory`
- `GovernanceSession` (v1.0+)
- public workflow CLI commands (v1.0+)

Not Covered by the Stability Promise

The following are not public API unless explicitly documented otherwise:

- anything under `aigc._internal`
- internal helper functions
- internal dataclass fields and lifecycle bits
- internal schema helper utilities
- dev-only scripts
- internal docs and design notes
- experimental adapters or modules explicitly labeled experimental

Host code must never import from `aigc._internal`.

Deprecation Policy

When a supported API is deprecated:

1. it remains functional for at least one documented deprecation window
2. the docs and changelog state the replacement path
3. warnings are emitted where practical
4. removal happens only in a major release unless there is a security or correctness emergency

Example:

- unified-mode opt-out behavior via `pre_call_enforcement=False` may remain deprecated in `1.x`, but any removal requires explicit release notes and a major-version decision unless a stronger exception is documented

---

Schema and DSL Compatibility

Policy DSL

- additive fields are preferred
- removals or incompatible meaning changes require major-version treatment

Audit artifacts

- additive evolution is required for existing invocation artifacts
- existing stable fields must not silently change meaning in `1.x`

Workflow evidence

- new workflow-level evidence may be added in `1.x`
- existing invocation artifact contracts must remain backward-compatible

---

CLI Compatibility

Documented CLI commands and documented flags are part of the public surface.

Rules:

- additive flags are allowed
- documented flags should not change semantics incompatibly in `1.x`
- removals require deprecation notice and migration guidance

Import Rule

Preferred imports come from:

```
from aigc import ...

Public module imports may also be documented where appropriate.

Do not import from:

from aigc._internal import ...

That path is private and may change at any time.

_____

Testing and Enforcement

The repo should enforce this contract through:
 • public API snapshot tests
 • docs parity checks
 • internal-import boundary checks
 • changelog and migration updates when public API changes

_____



---

# Draft file: `docs/MIGRATION_0_3_3_TO_1_0.md`

```md
# Migrating from AIGC `0.3.3` to `1.0.0`

This guide explains how to move from the current `v0.3.3` runtime to
`v1.0.0`.

AIGC `0.3.3` already provides:

- unified enforcement
- split enforcement
- workflow-aware evidence primitives
- provenance metadata
- `AuditLineage`
- `ProvenanceGate`
- `RiskHistory`
- split enforcement as the default decorator mode
```
```

AIGC `1.0.0` adds workflow-governance primitives while preserving the current
invocation-governance kernel.

Summary of What Changes

What stays the same
- `enforce_invocation()` remains supported
- `enforce_pre_call()` / `enforce_post_call()` remain supported
- `AIGC` remains the primary instance-scoped entry point
- `@governed` still defaults to split enforcement
- existing invocation artifact contracts remain valid
- host applications still own orchestration and model calls

What is new
- `GovernanceSession`
- workflow policy DSL fields
- workflow enforcement over step order, handoffs, and aggregate budgets
- agent identity/capability manifests
- workflow CLI/trace/export tooling
- optional Bedrock and A2A adapters
- optional escalation/review checkpoints

Migration Paths

Path A — Invocation-only users
If you only use invocation governance today and do not need workflow governance:

- you can stay on your current invocation pattern
- no mandatory code rewrite is required
- continue using public imports only
- read the public API stability doc and verify deprecation warnings

Path B — Multi-step host orchestration users
If your host already orchestrates multi-step agent workflows:

- introduce `GovernanceSession`
- map each governed step into a session step
- move sequence logic out of ad hoc host checks and into workflow policy
- add handoff constraints and budgets to the workflow DSL

Path C — Ecosystem users
If you use Bedrock multi-agent or A2A-style remote handoffs:

- adopt the relevant adapter
- map external agent identities into AIGC `AgentIdentity` /
  `AgentCapabilityManifest`
- keep transport/runtime in the host layer

- use AIGC only for governance decisions and evidence

Recommended Migration Sequence

1. Upgrade dependency to `1.0.0`
2. run the existing test suite unchanged
3. confirm invocation paths still pass
4. add workflow policies for one real workflow
5. wrap that workflow with `GovernanceSession`
6. add workflow trace/export checks to CI
7. expand to more workflows only after the first is stable

Split Enforcement Reminder

`0.3.3` already flipped `@governed` to `pre_call_enforcement=True` by default.

That remains the standard execution model in `1.0.0`.

No change required
If your call sites already:
- pass `pre_call_enforcement=True`, or
- rely on the default split behavior intentionally

Change required
If your call sites still rely on unified mode behavior, keep:
```python
@governed(..., pre_call_enforcement=False)

That remains an explicit compatibility path where still supported, but it is not
the recommended default.

_____

New Workflow Pattern

Before (0.3.3)

Host coordinates multiple governed calls manually:
 • call A
 • store artifact
 • call B
 • manually track sequence and budgets
 • manually correlate evidence

After (1.0.0)

Host still orchestrates, but AIGC provides workflow governance primitives:
 • open GovernanceSession
```

- enforce each step through session-aware policy
- correlate evidence automatically
- export workflow trace deterministically

Audit and Compliance Expectations

Invocation artifacts

No migration required for existing invocation artifact consumers.

Workflow evidence

If you adopt workflow governance:
- update downstream compliance/export consumers to read workflow correlation output
- preserve any existing invocation artifact ingestion unchanged

---

Docs to Update in Host Repos

When upgrading, update:
- internal integration docs
- CI commands if workflow tests are added
- import guidance to ensure no _internal imports remain
- operator docs for workflow tracing and export

Common Upgrade Risks

1. Treating AIGC as an orchestrator

Do not move orchestration logic into AIGC.

2. Mixing internal and public imports

Clean up any aigc._internal imports before or during migration.

3. Skipping workflow policy design

Do not re-create ad hoc workflow checks in host code if you want the benefits of 1.0.0.

4. Assuming adapters own the runtime

Bedrock/A2A adapters are translation layers, not runtime replacements.

Validation Checklist
 • existing invocation tests pass
 • no host imports from aigc._internal
 • split enforcement assumptions verified
 • workflow DSL validates
 • workflow trace/export works for at least one real workflow
 • operator docs updated
 • compliance/export consumers reviewed

_____

Draft additions for `RELEASE_GATES.md`

Append this after the existing `v0.3.3` section.

```md
---

## `v1.0.0` Groundwork Gate — PR-01

Branch:

- `feat/v1.0-01-adr-boundary-contract`

Checklist:

- [ ] `docs/decisions/ADR-0011-workflow-governance-v1.md` accepted or ready
      for acceptance review
- [ ] `docs/SDK_BOUNDARY.md` published
- [ ] `docs/PUBLIC_API_STABILITY.md` published
- [ ] `docs/MIGRATION_0_3_3_TO_1_0.md` drafted
- [ ] `README.md` and `PROJECT.md` updated to reflect the `v1.0.0` target
- [ ] no runtime behavior changed in PR-01
- [ ] no schema behavior changed in PR-01

---

## `v1.0.0` Capability Gates

### PR-02 — Workflow Audit Correlation Schema

- [ ] invocation artifact compatibility preserved
- [ ] schema change is additive only for existing invocation artifacts
- [ ] no new required fields on invocation artifacts
- [ ] workflow-level evidence contract documented
- [ ] schema and contract tests land with the change

### PR-03 — `GovernanceSession`
```

```
- [ ] `GovernanceSession` public API lands
- [ ] workflow/session lifecycle is covered by tests
- [ ] finalize behavior is deterministic
- [ ] no regression in invocation-only entry points
- [ ] no hidden orchestration logic introduced

### PR-04 — Workflow DSL Extension

- [ ] workflow fields validate through policy schema
- [ ] legacy policies remain valid
- [ ] composition semantics are documented
- [ ] invalid workflow policies fail deterministically

### PR-05 — Workflow Enforcement Engine

- [ ] step sequencing enforcement works
- [ ] cumulative budget enforcement works
- [ ] disallowed handoffs are blocked
- [ ] workflow-level fail-closed behavior is preserved
- [ ] invocation gate ordering remains unchanged

### PR-06 — Agent Identity and Capability Manifest

- [ ] agent identity schema validates
- [ ] capability mismatch fails closed
- [ ] manifest versioning rules are documented

### PR-07 — Bedrock Adapter

- [ ] Bedrock adapter is optional
- [ ] no hard AWS dependency added to core path
- [ ] supervisor/collaborator mapping is tested
- [ ] adapter docs clarify SDK boundary

### PR-08 — A2A Adapter

- [ ] A2A adapter is optional
- [ ] no transport ownership added to SDK
- [ ] agent-card and handoff normalization are tested
- [ ] adapter docs clarify SDK boundary

### PR-09 — Escalation / Human Review

- [ ] review-required state is enforceable
- [ ] approval/denial evidence is emitted
- [ ] pause/resume behavior is deterministic
- [ ] no manual bypass path exists

### PR-10 — Content Validation Hooks

- [ ] stable validator hook API lands
- [ ] core pipeline order is preserved
```

```
- [ ] hook failures are typed and auditable
- [ ] docs explain how this differs from full semantic guardrail products

### PR-11 — Workflow CLI / Trace / Export

- [ ] `aigc workflow trace` lands
- [ ] `aigc workflow export` lands
- [ ] `aigc workflow lint` lands
- [ ] fixture-based workflow export tests land

### PR-12 — Starter Workflow Policies

- [ ] starter policies validate by schema
- [ ] each starter has pass/fail coverage
- [ ] starters are documented as SDK adoption packs

### PR-13 — Integration Recipes / Quickstarts

- [ ] Bedrock recipe lands
- [ ] A2A recipe lands
- [ ] FastAPI recipe lands
- [ ] worker/queue recipe lands
- [ ] 15-minute quickstart is runnable

### PR-14 — Public API / Ops Hardening

- [ ] public API snapshot tests land
- [ ] supported environments matrix is documented
- [ ] operations runbook references are updated
- [ ] doc parity manifest updated for `v1.0.0`

---

## `v1.0.0` Final Release Gate

SDK boundary outcome:

- [ ] invocation governance remains intact
- [ ] workflow governance primitives are public and documented
- [ ] AIGC still does not own orchestration or model execution
- [ ] optional adapters do not collapse the provider-agnostic boundary

Adoption outcome:

- [ ] public API stability contract published
- [ ] migration guide published
- [ ] quickstart published
- [ ] starter workflow policies published
- [ ] integration recipes published

Documentation parity targets:
```

- [ ] `README.md`
- [ ] `PROJECT.md`
- [ ] `CHANGELOG.md`
- [ ] `docs/SDK_BOUNDARY.md`
- [ ] `docs/PUBLIC_API_STABILITY.md`
- [ ] `docs/MIGRATION_0_3_3_TO_1_0.md`
- [ ] `docs/PUBLIC_INTEGRATION_CONTRACT.md`
- [ ] `docs/AIGC_FRAMEWORK.md`
- [ ] `docs/architecture/AIGC_HIGH_LEVEL_DESIGN.md`
- [ ] `docs/architecture/ENFORCEMENT_PIPELINE.md`
- [ ] `schemas/audit_artifact.schema.json`
- [ ] `schemas/policy_dsl.schema.json`
- [ ] `doc_parity_manifest.yaml`

Release readiness:

- [ ] full CI passes
- [ ] packaging smoke passes
- [ ] public API snapshot passes
- [ ] workflow integration suite passes
- [ ] release notes drafted
- [ ] final changelog entry drafted
- [ ] release checklist reviewed

Fully complete doc parity checklist

This is based on the current parity model already in doc_parity_manifest.yaml, which today tracks README.md, PROJECT.md, docs/AIGC_FRAMEWORK.md, docs/architecture/AIGC_HIGH_LEVEL_DESIGN.md, and docs/architecture/ENFORCEMENT_PIPELINE.md.

A. Versioned values that must match everywhere
- version number
- release date
- test count
- coverage threshold
- audit schema version
- split enforcement default behavior
- current release scope summary

B. Canonical product identity
- package name: aigc-sdk
- import name: aigc
- repo name and branch references
- one-sentence product definition
- governance invariant language

C. SDK boundary statements

Must say the same thing in:

- README.md
- PROJECT.md
- docs/SDK_BOUNDARY.md
- docs/PUBLIC_INTEGRATION_CONTRACT.md
- docs/architecture/AIGC_HIGH_LEVEL_DESIGN.md

Boundary points:
- host owns orchestration
- host owns model calls
- host owns tool execution
- AIGC owns policy loading
- AIGC owns ordered governance checks
- AIGC owns audit generation
- AIGC does not become a platform

D. Invocation enforcement surface
- enforce_invocation
- enforce_pre_call
- enforce_post_call
- async parity references
- AIGC instance methods
- @governed default behavior
- unified opt-out wording

E. Workflow governance surface

Once added, these must be aligned across docs:
- GovernanceSession
- workflow DSL fields
- workflow trace/export CLI
- escalation/review wording
- optional Bedrock adapter wording
- optional A2A adapter wording

F. Public API stability language

Must align across:
- docs/PUBLIC_API_STABILITY.md
- README.md
- PROJECT.md
- CHANGELOG.md
- docs/PUBLIC_INTEGRATION_CONTRACT.md

Check:
- what is public
- what is internal
- deprecation window
- 1.x stability promise
- no _internal imports in user-facing docs

G. Policy DSL parity
- schemas/policy_dsl.schema.json

- policies/policy_dsl_spec.md
- docs/PUBLIC_INTEGRATION_CONTRACT.md
- README.md examples
- migration guide examples

Check:
- field names
- workflow DSL fields
- deprecation wording
- examples use real supported syntax

H. Audit artifact parity
- schemas/audit_artifact.schema.json
- README.md
- PROJECT.md
- docs/PUBLIC_INTEGRATION_CONTRACT.md
- docs/architecture/AIGC_HIGH_LEVEL_DESIGN.md
- migration guide
- changelog

Check:
- schema version
- required vs optional fields
- provenance fields
- workflow evidence wording
- one artifact per invocation attempt invariant

I. Canonical gate IDs

Must align with code and docs:
- guard_evaluation
- role_validation
- precondition_validation
- tool_constraint_validation
- schema_validation
- postcondition_validation
- risk_scoring

If workflow-level gate names are documented, add them deliberately and update parity tooling.

J. CLI parity
- aigc policy lint
- aigc policy validate
- aigc compliance export
- aigc workflow trace
- aigc workflow export
- aigc workflow lint

Check:
- command names
- flags
- examples

- availability by release

K. Quickstart and recipe parity
- installation commands
- editable install commands
- no-build-isolation guidance
- minimal hello-world example
- split-mode example
- workflow example
- Bedrock recipe
- A2A recipe

L. Repo map parity
- PROJECT.md tree
- new public modules listed
- new docs listed
- examples/recipes location listed
- no nonexistent files referenced

M. Release narrative parity
- README.md
- PROJECT.md
- CHANGELOG.md
- RELEASE_GATES.md
- migration guide

Check:
- 0.3.3 description is consistent
- 1.0.0 description is consistent
- no stale references to 0.3.2 as current
- no stale private-repo evolution wording leaks into public docs

N. Internal-doc exclusion list

Check doc_parity_manifest.yaml and keep updated:
- newly created internal docs excluded if appropriate
- newly created public docs not accidentally excluded
- wildcard paths still valid

O. Final parity signoff checklist
- all user-facing docs updated in same release wave
- doc parity script passes
- markdown lint passes
- examples compile or run
- public API snapshot passes
- no user-facing doc imports _internal
- release gates updated
- changelog updated
- migration guide updated

Recommended doc_parity_manifest.yaml expansion for v1.0

When you reach the PR that introduces the new docs, I would expand the parity_docs block from the current five-doc set to:

```
parity_docs:
  - README.md
  - PROJECT.md
  - docs/AIGC_FRAMEWORK.md
  - docs/SDK_BOUNDARY.md
  - docs/PUBLIC_API_STABILITY.md
  - docs/MIGRATION_0_3_3_TO_1_0.md
  - docs/PUBLIC_INTEGRATION_CONTRACT.md
  - docs/architecture/AIGC_HIGH_LEVEL_DESIGN.md
  - docs/architecture/ENFORCEMENT_PIPELINE.md
```

That is the smallest set that matches the new public contract cleanly, while staying close to the parity discipline already in develop.

If you want, I'll turn this into a copy-paste PR-01 commit payload next, with the exact markdown for all five files and the RELEASE_GATES.md patch in repo-ready form.